

Compliance Audit

ISO/IEC 25010 - Maintainability

Modularity

Beschrijving

Volgens het ISO20510 betekent Modularity de mate waarin een systeem of computerprogramma is samengesteld uit afzonderlijke componenten, zodat een wijziging in één component minimale gevolgen heeft voor andere componenten.

Toepassing op dit systeem

Het systeem is opgebouwd uit verschillende componenten die allemaal hun eigen taak hebben. Dit is opgesplitst in de ontwikkel- en development omgeving.

-Ontwikkelmodule(python project)

De pythoncode voor dit project wordt lokaal ontwikkeld. Deze code bestaat uit afzonderlijke modules, zoals engagement.py, classifieer.py en metadata.py. Dit zorgt voor een scheiding tussen de functies, waardoor ze makkelijk aanpasbaar zijn zonder gevolgen voor de rest van de code.

Het versiebeheer vindt plaats in GitLab. Dit maakt het makkelijk om wijzigingen te beheren en indien nodig ook terug te draaien.

-Build- en deployment module

Na elke push zorgt GitLab er automatisch voor via een pipeline, dat de code wordt verwerkt tot een Docker Image. Deze wordt vervolgens gezet op de GitHub Container Registry. Hier worden alle versies van de image beheert, ongeacht waar de image op draait.

-Uitvoermodule

De serveromgeving haalt dagelijks de nieuwste image van de GitHub Container Registry en voert deze uit. De volgende taken worden uitgevoerd:

- Ophalen van metadata en in opslaan in de feiten tabel.
- Aanmaken van de juiste dimensies
- Berekenen en opslaan van het sentiment en engagement

Door gebruik te maken van een Docker image maakt het niet uit op welke omgeving de image gerund wordt.

-Data laag

De data wordt opgeslagen in een PostgreSQL-database. De structuur van de database is gemaakt op basis van een sterdiagram.

Door de data op te splitsen in verschillende dimensies, is het aanpassen van 1 tabel heel erg makkelijk en heeft weinig invloed op de andere tabellen.

-Rapportagemodule

Uiteindelijk wordt de data weergegeven op een dashboard in Looker Studio. Dit heeft verder geen invloed op de data zelf, want de data wordt alleen uitgelezen en niet bewerkt.

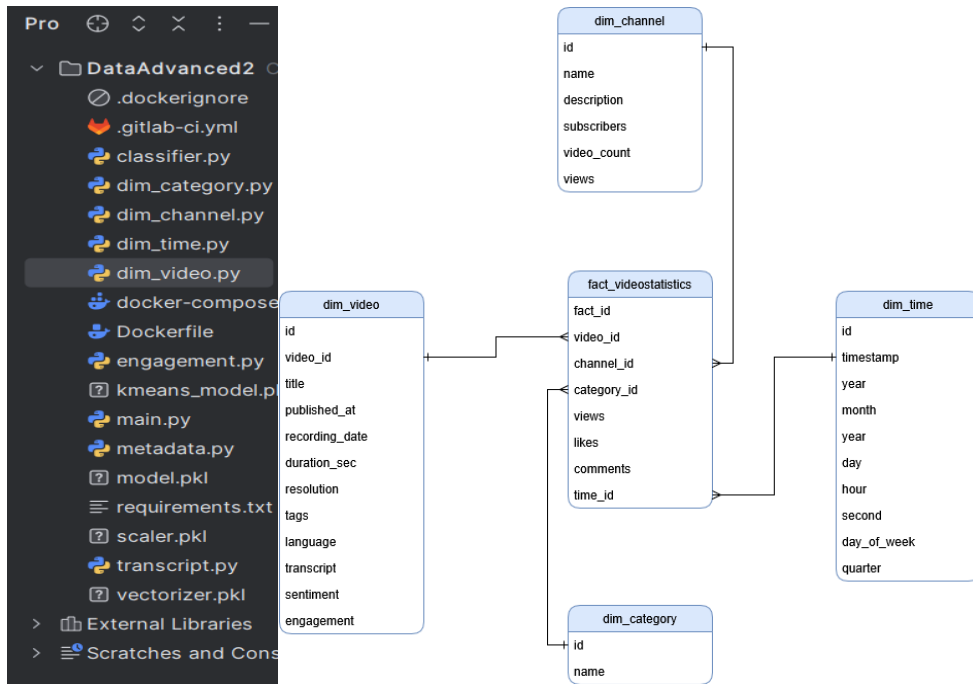
Conclusie

Door de modulaire opzet, kan ik de modules gemakkelijk aanpassen zonder invloed op de rest van de componenten. Zo kan ik de pythoncode gemakkelijk aanpassen in de ontwikkelmodule. Dit heeft geen invloed op de rest van de componenten totdat ik de code heb gepusht naar GitLab.

Beoordeling

Het systeem voldoet in hoge mate aan de ISO/IEC 25010-criteria voor Modularity binnen het kader van Modularity. De architectuur is opgebouwd uit duidelijk gescheiden componenten: de analysemodule, CI/CD-pipeline, uitvoeromgeving, data laag en rapportagelaag. Elke laag heeft zijn eigen verantwoordelijkheid. Wijzigingen in één onderdeel, zoals aanpassingen aan het sentimentanalysemodel of de build-pipeline, hebben geen directe invloed op andere onderdelen.

Bewijsmateriaal



CI/CD Catalog Help Editor accessibility guide

```
1 stages:
2 | - build-and-push
3
4 build-and-push:
5 | image: docker:24.0.5
6 | stage: build-and-push
7 | services:
8 | | - docker:24.0.5-dind
9 | script:
10 | - docker build -t s1156206-db-logger:latest .
11 | - docker tag s1156206-db-logger:latest ghcr.io/s1156206/data-advanced:latest
12 | - echo $GHC_KEY | docker login ghcr.io -u s1156206 --password-stdin
13 | - docker push ghcr.io/s1156206/data-advanced:latest
14
15
16
```

Reusability

Beschrijving

Volgens het ISO25010 betekent Reusability de mate waarin een product als asset in meer dan één systeem kan worden gebruikt, of bij het bouwen van andere assets.

Toepassingen op dit systeem

De code is opgebouwd uit verschillende modules, zoals `classifier.py` en `engagement.py`. Dit maakt het project erg herbruikbaar, omdat deze modules gemakkelijk in een ander project gebruikt kunnen worden.

De GitLab CI/CD pipeline is ook erg herbruikbaar voor een nieuw project. Het enige wat aangepast zou moeten worden, is de naam van de repository.

Door gebruik te maken van een image, betekent dit dat het project in meerdere omgevingen opgezet kan worden.

De database is ingericht met een sterschema. De database is wel volledig ingericht voor dit project, wat herbruikbaarheid lastiger maakt. Doordat de database is ingericht met behulp van een sterschema, zouden tabellen echter wel hergebruikt kunnen worden.

Het Looker Studio project is niet herbruikbaar. De data wordt opgehaald met hele specifieke queries uit de database. Om dezelfde tabellen / grafieken te kunnen vullen, heb je precies dezelfde database nodig als die ik nu heb.

Conclusie

Het systeem bestaat voor een groot deel uit herbruikbare componenten, zoals de python modules, pipeline en de database. Het dashboard is echter niet herbruikbaar, door de specifieke queries die gebruikt worden om de tabellen te vullen

Beoordeling

Het systeem voldoet voor het grootste deel aan de ISO/IEC 25010-criteria voor Reusability. Het enige onderdeel dat niet aan deze eis voldoet, is het dashboard. Deze is specifiek opgebouwd voor het huidige project en kan niet hergebruikt worden voor andere projecten.

Bewijsmateriaal

Aangepaste query opgeven

```
1 WITH base AS (  
2   SELECT  
3     video_id,  
4     channel_id,  
5     views,  
6     likes,  
7     collected_at::date AS collected_date,  
8     DATE_TRUNC('week', collected_at)::date AS week_start  
9   FROM fact_videostatistics  
10 ),  
11 latest AS (  
12   SELECT DISTINCT ON (video_id, week_start)  
13     video_id,  
14     channel_id,  
15     week_start,  
16     views AS views_now,  
17     likes AS likes_now,  
18     collected_date  
19   FROM base  
20   ORDER BY video_id, week_start, collected_date DESC  
21 ),  
22 previous AS (  
23   SELECT DISTINCT ON (video_id, week_start)  
24     video_id,  
25     week_start,  
26     views AS views_prev,  
27     likes AS likes_prev  
28   FROM base  
29   ORDER BY video_id, week_start, collected_date DESC  
30 )  
31 SELECT  
32   cur.video_id,  
33   cur.channel_id,  
34   c.name AS channel_name,  
35   ... AS ...  
36
```

Analysability

Beschrijving

Volgens het ISO25010 betekent Analysability de mate van effectiviteit en efficiëntie waarmee het mogelijk is om de impact op een product of systeem van een beoogde wijziging aan een of meer onderdelen te beoordelen, een product te diagnosticeren op tekortkomingen of oorzaken van storingen, of onderdelen te identificeren die moeten worden gewijzigd.

Toepassing op het systeem

De CI/CD pipeline build de code om tot een image. Het bouwen van deze image lukt ook als er errors zijn in de code. Je kunt echter wel zien of je fouten hebt gemaakt in de Dockerfile zelf. Door het toevoegen van een test job zou je kunnen voorkomen dat een image gebouwd wordt als er errors in zitten.

De prints in de code en errors worden allemaal gelogd in de docker.out file op de server. Er kan dus altijd makkelijk teruggekeken worden wat je code heeft uitgevoerd en waar het eventueel mis is gegaan tijdens het runnen.

In de logs is ook een tijdstip verwerkt. Je kunt dus altijd zien wanneer je code is uitgevoerd. Het enige wat ontbreekt is de mogelijkheid om te kijken wie de code heeft uitgevoerd.

Conclusie

Het systeem maakt geen gebruik van een test-stage in de pipeline. Verder maakt het wel gebruik van een output file, waar print statements en errors te zien zijn.

Beoordeling

Het systeem voldoet voor een deel aan de ISO/IEC 25010-criteria voor Analysability. Errors kunnen gezien worden in een output file op de server, nadat het script gerund heeft. Er is dus geen mogelijkheid om vooraf te testen of de code werkt of niet. Door een teststage toe te voegen aan de pipeline, zou het systeem verbeterd kunnen worden.

Verder zou een goede toevoeging zijn om een gebruiker toe te voegen aan de logfiles. Op deze manier kun je zien wie het script heeft gerund.

Bewijsmateriaal

```
prod Error Head "https://ghcr.io/v2/s1156206/data-advanced/manifests/latest": unauthorized
Error response from daemon: Head "https://ghcr.io/v2/s1156206/data-advanced/manifests/latest": unauthorized
Sun 14 Sep 18:16:05 UTC 2025
prod Pulling
prod Pulled
Network s1156206_default Creating
Network s1156206_default Created
Container s1156206-db-logger Creating
Container s1156206-db-logger Created
Attaching to s1156206-db-logger
s1156206-db-logger | [youtube] Extracting URL: https://www.youtube.com/watch?v=eifBCQ871l8
s1156206-db-logger | Transcript voor XXWrzrY4W7E: gevonden
s1156206-db-logger | Transcript voor WKb9pGFy-fw: gevonden
s1156206-db-logger | Transcript voor FEqZyVT6dSo: gevonden
s1156206-db-logger | Transcript voor vjWKG_JE32E: gevonden
s1156206-db-logger | Transcript voor KTbeNNUOmVI: gevonden
s1156206-db-logger | Transcript voor uW98YqvLeKo: gevonden
s1156206-db-logger | Transcript voor MlUSMibsB18: gevonden
s1156206-db-logger | Transcript voor 9YjCbS22WtE: gevonden
s1156206-db-logger | Transcript voor QIe5zg4Uof4: gevonden
s1156206-db-logger | Transcript voor z7TyPa8JbG8: gevonden
s1156206-db-logger | Transcript voor BEOfgSYX3XA: gevonden
s1156206-db-logger | Transcript voor nLMJkNiuPSc: gevonden
s1156206-db-logger | Transcript voor WsHfnJLXP-U: gevonden
s1156206-db-logger | Transcript voor GBxFrTBjJGs: gevonden
s1156206-db-logger | Transcript voor MENMuLYvZqs: gevonden
```

Modifiability

Beschrijving

Volgens het ISO25010 betekent Modifiability de mate waarin een product of systeem effectief en efficiënt kan worden aangepast zonder dat er fouten worden geïntroduceerd of de bestaande productkwaliteit afneemt.

Toepassing op het system

Het systeem maakt gebruik python-libraries. De libraries verminderen de complexiteit van de code die zelf geïmplementeerd moet worden. De gebruikte libraries zijn bekende libraries, zoals Pandas en SQLAlchemy. Deze libraries worden vaak geüpdatet door grote communities, wat de kans op fouten bij hergebruik verkleint.

Daarnaast maakt het systeem gebruik van duidelijke programmeerconventies, zoals comments om aan te geven wat elk stuk code uitvoert.

De pipeline is volledig geautomatiseerd. Menselijke fouten kunnen hierdoor niet gemaakt worden in dit deel van het systeem.

Door het gebruik van Git versiebeheer, kunnen fouten eenvoudig worden teruggedraaid. Fouten die optreden kunnen gemakkelijk worden gezien in de docker.out file. De fouten kunnen op die manier snel worden opgespoord.

Conclusie

Het systeem maakt gebruik van python-libraries voor eenvoudigere code en minder kans op fouten. Verder wordt er gebruik gemaakt van programmeerconventies. Fouten kunnen makkelijk worden opgespoord met de docker.out file en hersteld worden door het Git versiebeheer.

Beoordeling

Het systeem voldoet in aanzienlijke mate aan de ISO/IEC 25010-criteria voor Modifiability. De combinatie van versiebeheer, CI/CD-automatisering en gestructureerde code maakt het mogelijk om veilig en gecontroleerd wijzigingen aan te brengen. De modifiability van het systeem kan verder worden verbeterd door het toevoegen van automatische teststappen aan de CI/CD-pipeline.

Bewijsmateriaal

```
#Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(engagement[features])

X_scaled_df = pd.DataFrame(X_scaled, columns=features)
X_scaled_df['video_id'] = engagement['video_id']

#k-means clustering
kmeans = KMeans(n_clusters=3, random_state=42)
X_scaled_df['Cluster'] = kmeans.fit_predict(X_scaled_df[features])

#Engagement per cluster berekenen
cluster_means = engagement.groupby(X_scaled_df['Cluster'])[['likes_per_view', 'comments_per_view']].mean()

#Clusters sorteren op like-ratio (laag -> hoog)
#Cluster getal is altijd random, dus vandaar even kijken welk getal welk label krijgt
sorted_clusters = cluster_means['likes_per_view'].sort_values().index.tolist()

labels = ['Low Engagement', 'Medium Engagement', 'High Engagement']
cluster_to_label = {sorted_clusters[i]: labels[i] for i in range(len(sorted_clusters))}

#Labels toevoegen
X_scaled_df['Engagement Cluster'] = X_scaled_df['Cluster'].map(cluster_to_label)

#Opslaan
with engine.begin() as connection:
```

```
psycog2-binary
moviepy
google-api-python-client
requests
yt_dlp
isodate
pandas
pypickle
scikit-learn
numpy
SQLAlchemy
```

Testability

Beschrijving

Volgens het ISO25010 betekent Testability de mate van effectiviteit en efficiëntie waarmee testcriteria voor een systeem, product of onderdeel kunnen worden vastgesteld en tests kunnen worden uitgevoerd om te bepalen of aan die criteria is voldaan.

Toepassing op het system

De huidige CI/CD pipeline bouwt automatisch een Docker image van de applicatie, maar voert geen automatische tests uit voordat de image wordt gepubliceerd. Er is daardoor geen ingebouwd mechanisme om vooraf te zien of de Python-code foutloos functioneert.

Wel wordt tijdens het runnen van de code alle print statements en foutmeldingen opgeslagen in de docker.out file op de server. Deze logging maakt het mogelijk om na afloop de werking van het systeem te analyseren en te controleren of de resultaten correct zijn.

Daarnaast ondersteunt de modulaire opbouw van de code het uitvoeren van afzonderlijke unittests, mocht een testfase worden toegevoegd in de toekomst.

Conclusie

Het systeem biedt mogelijkheden voor handmatige validatie via logging, maar bevat geen automatische testfase binnen de pipeline. De huidige opzet ondersteunt beperkte testbaarheid: fouten kunnen pas na uitvoering worden gedetecteerd. Door het toevoegen van een test-stage in de CI/CD-pipeline kan de testbaarheid aanzienlijk worden verbeterd.

Beoordeling

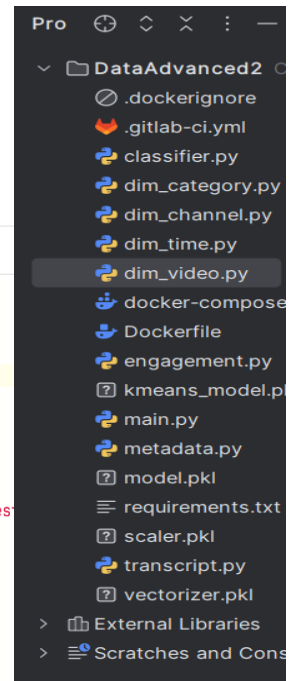
Het systeem voldoet weinig aan de ISO/IEC 25010-criteria voor Testability.

De aanwezige logging biedt een vorm van foutdetectie, maar geautomatiseerde testen ontbreken. Door gebruik te maken van een testfase in de CI/CD-pipeline kan de testbaarheid worden verhoogd en kunnen fouten eerder in het ontwikkelproces worden opgespoord.

Bewijsmateriaal

CI/CD Catalog Help Editor accessibility guide

```
1 stages:
2 | - build-and-push
3
4 build-and-push:
5 | image: docker:24.0.5
6 | stage: build-and-push
7 | services:
8 | | - docker:24.0.5-dind
9 | script:
10 | | - docker build -t s1156206-db-logger:latest .
11 | | - docker tag s1156206-db-logger:latest ghcr.io/s1156206/data-advanced:latest
12 | | - echo $GHCR_KEY | docker login ghcr.io -u s1156206 --password-stdin
13 | | - docker push ghcr.io/s1156206/data-advanced:latest
14
15
16
```



ISO/IEC 27000

Beschrijving

De ISO 27000-serie normen zijn een reeks van internationale standaarden die betrekking hebben op informatiebeveiliging. Deze normen zijn ontwikkeld door de International Organization for Standardization (ISO) en bevatten richtlijnen voor het ontwerpen, implementeren en beheren van beveiligingsmaatregelen om de vertrouwelijkheid, integriteit en beschikbaarheid van informatie te waarborgen.

Toepassing op het systeem

Het systeem maakt gebruik van containerisering met behulp van Docker. De python code is verpakt in een Docker image. Dit zorgt ervoor dat de code is geïsoleerd van de rest van de omgeving en hier dus geen invloed op kan uitoefenen.

Elke avond wordt deze image opgehaald van de GitHub Container Registry. Deze wordt opgehaald via een beveiligde HTTPS verbinding en wordt daarna uitgevoerd in de Docker omgeving. De data wordt dus met behulp van encryptie verstuurd.

De opgehaalde metadata wordt veilig opgeslagen na het runnen van de code via beveiligde internetverbinding binnen dezelfde hostomgeving.

Verder zijn wachtwoorden, api-keys en gebruikersnamen opgeslagen in omgevingsvariabelen. Mocht de image dus ooit uitlekken, dan is de docker-compose.yml file hier niet in opgeslagen en zijn de wachtwoorden niet zichtbaar. Ook in GitLab wordt er gebruik gemaakt van omgevingsvariabelen.

Looker Studio maakt verbinding met de database via een beveiligde verbinding. Het dashboard kan verder ook alleen de data lezen en het dus niet veranderen.

Daarnaast wordt het systeem beheerd via GitLab versiebeheer, waardoor elke wijziging in de code wordt vastgelegd. Hierdoor is altijd duidelijk wie welke aanpassingen heeft gedaan en kan bij fouten eenvoudig worden teruggedraaid naar een vorige versie.

Om toegang te krijgen tot het interne netwerk wordt gebruik gemaakt van een Virtual Private Network(VPN). Om te verbinden moet je inloggen met een gebruikersnaam en wachtwoord. Dit creëert een versleutelde verbinding tussen de ontwikkelomgeving en het netwerk. Hierdoor is het niet mogelijk om rechtstreeks verbinding te maken met de server zonder geldige VPN toegang. Dit verhoogt de beveiliging bij gegevensuitwisseling en voorkomt dat onbevoegden toegang krijgen tot interne systemen.

Conclusie

Het systeem maakt gebruik van een veilige en gescheiden infrastructuur waarin het delen van data uitsluitend via beveiligde verbindingen verloopt. De combinatie van Docker isolatie, gebruik van omgevingsvariabelen en beperkte toegang tot de database zorgt voor een sterke basis in gegevensbescherming.

Beoordeling

Het systeem voldoet grotendeels aan de ISO/IEC 27000 criteria. Verbeterpunten liggen met name op het gebied van monitoring en logging van toegangspogingen. Door aanvullende maatregelen te implementeren, zoals periodieke kwetsbaarheidsscans kan de beveiliging verder worden versterkt.

Algemene Verordening Persoonsgegevens(AVG)

Beschrijving

De Algemene Verordening Gegevensbescherming (AVG) is de strengste privacy- en beveiligingswet ter wereld. Hoewel deze is opgesteld en aangenomen door de Europese Unie (EU), legt deze verplichtingen op aan organisaties overal ter wereld, zolang ze zich richten op of gegevens verzamelen van mensen in de EU. De verordening is op 25 mei 2018 in werking getreden. De AVG zal zware boetes opleggen aan degenen die de privacy- en beveiligingsnormen overtreden, met boetes die kunnen oplopen tot tientallen miljoenen euro's.

Toepassing op het systeem

Het systeem maakt alleen gebruik van openbare gegevens, afkomstig van Youtube. Er worden bijvoorbeeld video- en kanaalstatistieken opgehaald. Alle gegevens zijn afkomstig van de Youtube Api die zelf voldoet aan de AVG en alleen openbare gegevens deelt die geen gevoelige persoonsinformatie bevatten.

De verwerkte data wordt opgeslagen in een PostgreSQL database binnen een beveiligde netwerk omgeving. Deze is alleen toegankelijk voor geautoriseerde gebruikers.

Alle data wordt opgevraagd of verstuurd via HTTPS, waardoor de hele weg versleuteld is.

Verder is het dashboard in Looker Studio read-only. Dit betekent dat er via het dashboard geen aanpassingen kunnen worden gemaakt aan de data.

Wanneer een video wordt verwijderd, zal deze langzaam verdwijnen van het dashboard. Echter verdwijnt deze niet uit de database. De AVG is alleen van sprake bij persoonsgegevens. De metadata uit een video behoort hier niet tot. Dus ook als de video verwijderd wordt, mag de data gebruikt blijven worden.

Indien 'right to be forgotten' toch op één of andere manier van sprake is, kan dit makkelijk geïmplementeerd worden. Indien de opgevraagde metadata niet meer beschikbaar is (de video is dus verwijderd) kan alle metadata in het systeem ook verwijderd worden.

Conclusie

Het systeem voldoet aan de eisen van de AVG, aangezien er geen gebruik wordt gemaakt van persoonsgegevens, de data afkomstig is uit publieke bronnen en de dataopslag gebeurt binnen een beveiligde omgeving.

Beoordeling

Het systeem voldoet aan de AVG-principes van dataminimalisatie, veiligheid van verwerking en transparantie. Er worden namelijk geen persoonsgegevens of gevoelige informatie verwerkt. Eventuele risico's zijn minimaal, maar kunnen verder worden beperkt door periodiek te controleren of de dataset geen velden bevat die indirect herleidbaar zijn tot personen (bijvoorbeeld gebruikersnamen).